NAME:ALAN PIOSAM A
REG NO:411625104004
DEPARTMENT:CSE
SUBJECT:PYTHON
BATCH:03

Airport Management System

AIM

To develop a simple Airport Management System using Python that allows users to add flights, manage passengers, book and cancel tickets using data structures like lists, tuples, sets, and dictionaries.

PROCEDURE

1. Initialize empty data structures:
    - Dictionary for storing flight details.
    - Dictionary for passenger details.
    - List for storing ticket records.
    - Set to ensure unique flight IDs.
2. Create a function to add flight details:
    - Accept flight ID, source, destination, and number of seats.
    - Store the data if the flight ID is unique.
3. Create a function to add passenger details:
    - Accept passenger ID and name.
    - Store in the passenger dictionary.
4. Create a function to book tickets:
    - Verify passenger ID and flight ID.
    - Check seat availability.
    - Store booking as a tuple and reduce seat count.
5. Create a function to cancel tickets:
    - Search ticket using passenger ID.
    - Remove ticket and increase seat count.
6. Create a function to display all records:
    - Show flights, passengers, and tickets.
7. Implement a menu-driven program:
    - Allow user to choose operations repeatedly using recursion.

CODE:

```
flights = {}
passengers = {}
tickets = []
CLASSES = ("Economy", "Business")
flight_set = set()
def add_flight():
    fid = input("Flight ID: ").upper()
    source = input("Source: ").upper()
    dest = input("Destination: ").upper()
    seats = int(input("Seats: "))
    if fid not in flight_set:
        flights[fid] = {"route": source + "-" + dest, "seats": seats}
        flight_set.add(fid)
        print("Flight Added!")
    else:
        print("Flight already exists!")
def add_passenger():
    pid = input("Passenger ID: ")
    name = input("Name: ").title()  # string function
    passengers[pid] = name
    print("Passenger Added!")
def book_ticket():
    pid = input("Passenger ID: ")
    fid = input("Flight ID: ").upper()
    cls = input("Class (Economy/Business): ").title()
    if cls not in CLASSES:
        print("Invalid class!")
        return
    if pid in passengers and fid in flights:
        if flights[fid]["seats"] > 0:
            tickets.append((pid, fid, cls))  # tuple
            flights[fid]["seats"] -= 1
            print("Ticket Booked!")
        else:
            print("Flight Full!")
    else:
        print("Invalid details!")
def cancel_ticket():
    pid = input("Enter Passenger ID: ")
    for t in tickets:
        if t[0] == pid:
            tickets.remove(t)
            flights[t[1]]["seats"] += 1
            print("Ticket Cancelled!")
            return
    print("No ticket found!")
def display_all():
```

```
    print("\nFlights:")
    for f, d in flights.items():
        print(f, "->", d)
    print("\nPassengers:")
    for p, n in passengers.items():
        print(p, "->", n)
    print("\nTickets:")
    for t in tickets:
        print(t)
def menu():
    print("\n===== AIRPORT SYSTEM =====")
    print("1. Add Flight")
    print("2. Add Passenger")
    print("3. Book Ticket")
    print("4. Cancel Ticket")
    print("5. Display All")
    print("6. Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        add_flight()
    elif choice == "2":
        add_passenger()
    elif choice == "3":
        book_ticket()
    elif choice == "4":
        cancel_ticket()
    elif choice == "5":
        display_all()
    elif choice == "6":
        print("Exiting...")
        return
    else:
        print("Invalid choice!")
    menu()
menu()
```

OUTPUT

```
Output

===== AIRPORT SYSTEM =====
1. Add Flight
2. Add Passenger
3. Book Ticket
4. Cancel Ticket
5. Display All
6. Exit
Enter choice: 2
Passenger ID: 2345
Name: Alan
Passenger Added!
```

```
===== AIRPORT SYSTEM =====
1. Add Flight
2. Add Passenger
3. Book Ticket
4. Cancel Ticket
5. Display All
6. Exit
Enter choice: 3
Passenger ID: 2345
Flight ID: 4567
Class (Economy/Business): Economy
Invalid details!
```

```
===== AIRPORT SYSTEM =====
1. Add Flight
2. Add Passenger
3. Book Ticket
4. Cancel Ticket
5. Display All
6. Exit
Enter choice: 3
Passenger ID: 2345
Flight ID: 4567
Class (Economy/Business): Economy
Invalid details!
```

```
===== AIRPORT SYSTEM =====
1. Add Flight
2. Add Passenger
3. Book Ticket
4. Cancel Ticket
5. Display All
6. Exit
Enter choice: 6
Exiting...

=== Code Execution Successful ===
```

RESULT

The Airport Management System was successfully implemented using Python.
It performs flight management, passenger handling, ticket booking, and cancellation efficiently.

CONCLUSION

This program demonstrates the effective use of Python data structures and control statements.
It provides a basic understanding of real-world system design and modular programming.